

Primer Entrega del Proyecto semestral 2026-2

Análisis y Diseño de Algoritmos

Aceleración de una ruta bajo presupuesto y cobertura mínima de auditoría

Definición del problema, NP-dificultad
y eficiencia de las implementaciones en Python y C++

Integrantes:

Michael Ramirez — 63421 — Grupo 03
Manuela Vallejo Toro — 28239 — Grupo 01

Universidad de Caldas

Docente:
Luz Enith Guerrero

Noviembre de 2026

ÍNDICE

1. El problema **1**
1.1. Aceleración de la cadena 1
1.2. Cobertura mínima de auditoría 2

2. Qué significa NP-difícil **2**

3. Por qué este problema es NP-difícil **2**

4. Eficiencia en tiempo y en espacio **4**
4.1. Mochila 0/1 4
4.2. Set cover voraz 6
4.3. El óptimo de set cover, solo en Python 6

5. Qué implementación conviene optimizar **7**

6. Validación experimental **8**
6.1. Mochila 8
6.2. Set cover 10
6.3. La misma prueba en una segunda máquina 11

7. Cierre **14**

1. El problema

Hay un flujo que atiende cada solicitud pasando por n etapas en serie. La etapa i dura t_i si no se toca. Se puede acelerar: cuesta c_i y baja su duración en d_i. El presupuesto para acelerar es B. Por otro lado, cada etapa cubre algunas categorías de riesgo si se audita. Auditar todas sale caro, así que solo se audita un subconjunto, pero entre las elegidas tiene que aparecer cada una de las r categorías al menos una vez.

Este informe se queda en esas dos decisiones por separado, sobre la cadena en serie, que es lo que resuelven los programas del repositorio. El enunciado pide además tres variantes: riesgos con peso distinto, un solo presupuesto para acelerar y auditar a la vez, y etapas en paralelo (un grafo dirigido acíclico, donde la duración ya no es la suma de todas las etapas). Esas variantes cambian el modelo. No cambian el argumento de NP-dificultad de la cadena, que es el caso particular del que se parte.

1.1. Aceleración de la cadena

Entrada. El número de etapas n, el presupuesto B, y para cada etapa la terna (t_i, c_i, d_i): duración normal, costo de acelerarla y reducción de tiempo si se acelera. En las instancias de este proyecto esos números salen del generador: cada costo está entre 5 y 18, y B es el 35 % de la suma de los costos, redondeado hacia abajo. B crece linealmente con n. Eso importa cuando se contraste el tiempo medido con la función teórica.

Salida. Un vector $x = (x_1, \ldots, x_n)$ con cada $x_i \in \{0, 1\}$. Si $x_i = 1$, la etapa i se acelera.

Objetivo. Minimizar la duración total de la cadena sin pasarse de B:

$$\text{mín} \sum_{i=1}^{n} (t_i - d_i x_i) \quad \text{sujeto a} \quad \sum_{i=1}^{n} c_i x_i \leq B, \quad x_i \in \{0, 1\}. \tag{1}$$

Como las etapas están en serie, todas están en el camino crítico. La duración total es la suma de las duraciones, aceleradas o no. La suma de las t_i no depende de qué etapas se elijan:

$$\sum_{i=1}^{n} (t_i - d_i x_i) = \sum_{i=1}^{n} t_i - \sum_{i=1}^{n} d_i x_i. \tag{2}$$

El primer sumando es constante. Minimizar la duración es, exactamente, maximizar la suma de las reducciones d_i de las etapas aceleradas, con la suma de sus costos c_i a lo más B. Eso es la mochila 0/1: el valor del objeto i es d_i y su peso es c_i. No es la mochila fraccionaria ni la mochila en la que el mismo objeto puede entrar varias veces. Cada etapa se acelera una vez o no se acelera.

Tamaño de la entrada. Para la programación dinámica el tamaño que hace crecer el trabajo es el par (n, B): n filas y B columnas. Hay que separar eso del tamaño de la codificación. B entra escrito en binario, con unos $\log_2 B$ bits. El tamaño del archivo no es proporcional a B; es proporcional a n más el número de bits de los números. La sección 4 usa las dos medidas, porque de ahí sale que el algoritmo es pseudo-polinomial.

1.2. Cobertura mínima de auditoría

Entrada. n interacciones candidatas a auditar y r categorías de riesgo. De cada interacción se conoce el subconjunto de categorías que cubriría. En el generador, $r = \lfloor 0.8n \rfloor$, y cada etapa cubre entre 2 y 5 riesgos elegidos al azar.

Salida. Un subconjunto de las interacciones. El objetivo es que ese subconjunto sea lo más pequeño posible y que la unión de sus riesgos sea el universo completo de r categorías. Es el problema de cobertura de conjuntos (*set cover*). El algoritmo del repositorio no minimiza el costo de auditar: minimiza el número de auditorías. La versión con costos o con importancia distinta es la variante ponderada del enunciado. Aquí se trabaja la versión sin pesos, que es la implementada.

2. Qué significa NP-difícil

Hace falta un problema de decisión: la respuesta es sí o no. «¿Existe un subconjunto que sume exactamente t?» es de decisión. «¿Cuál subconjunto maximiza el valor?» es de optimización. NP-completo se reserva para los de decisión. Un problema de optimización se llama NP-difícil cuando es, por lo menos, tan difícil como los NP-completos.

P es la clase de los problemas de decisión que se resuelven en tiempo polinomial en el tamaño de la entrada. NP es la clase de los problemas de decisión en los que todo sí tiene un certificado corto que se verifica en tiempo polinomial. NP no quiere decir «no polinomial». Verificar una solución propuesta puede ser fácil aunque encontrarla no lo sea. Para la mochila, el certificado es la lista de objetos elegidos: se suma su peso, se suma su valor y se comparan contra B y contra el umbral. Eso es lineal.

La comparación entre problemas se hace con una reducción de Karp, o reducción polinomial *many-one*. A se reduce a B, y se escribe $A \leq_p B$, si existe una transformación f, calculable en tiempo polinomial, tal que la instancia x es un sí de A exactamente cuando $f(x)$ es un sí de B. Si alguien resolviera B en tiempo polinomial, componiendo con f resolvería A también en tiempo polinomial. La reducción dice que B es al menos tan difícil como A. El sentido importa: se reduce el problema ya conocido al problema nuevo, no al revés.

B es NP-difícil si todo problema de NP se reduce polinomialmente a B. B es NP-completo si además B está en NP. Karp demostró en 1972 que 21 problemas de combinatoria son NP-completos, encadenando reducciones desde Satisfiability. En esa lista están, con los nombres que él usa, Knapsack y Set Covering [3].

3. Por qué este problema es NP-difícil

El Knapsack de Karp no es todavía la mochila con valor y peso por separado. Su enunciado es: dados enteros positivos $a_1, \ldots, a_m$ y un entero t, ¿existe un subconjunto que sume exactamente t? Hoy ese problema se llama *subset sum*, o suma de subconjuntos. Karp lo prueba NP-completo por reducción desde Exact Cover. Exact Cover pregunta si, de una familia de m conjuntos sobre un universo de p elementos, se puede escoger una subfamilia que cubra cada elemento una sola vez.

La construcción se puede revisar sin tomar la cita como caja negra. Si los conjuntos son $S_1, \ldots, S_m$ y el universo es $\{1, \ldots, p\}$, se pasa a base $\beta = m + 1$ y se define

$$a_j = \sum_{i \in S_j} \beta^{i-1}, \qquad \tau = \sum_{i=1}^{p} \beta^{i-1}, \qquad \beta = m + 1. \tag{3}$$

a_j es el número del conjunto S_j y τ es el objetivo de la suma. En base β, el dígito i de a_j vale 1 si el elemento i está en S_j y 0 si no. Un subconjunto de los a_j suma τ si y solo si, en cada posición, los dígitos suman exactamente 1. Como cada dígito aparece en a lo más m conjuntos y $\beta = m+1$, esa suma no produce acarreo: el dígito i del total es precisamente cuántos conjuntos elegidos contienen al elemento i. Que todos los dígitos valgan 1 es la definición de cobertura exacta.

La transformación es polinomial. Cada a_j tiene p dígitos en base β, así que se escribe con $O(p \log m)$ bits. El valor es grande; la codificación binaria no lo es. El tamaño de la instancia de suma de subconjuntos queda polinomial en el tamaño de la instancia de cobertura exacta. Esa es la reducción de Karp:

$$\textsc{Exact Cover} \leq_p \textsc{Knapsack}.$$

De ahí a la mochila 0/1 el paso ya es directo y se puede hacer a mano. Se toma la instancia $(a_1, \ldots, a_n, t)$ de suma de subconjuntos y se fabrica una mochila con n objetos, donde el objeto i tiene peso $c_i = a_i$ y valor $d_i = a_i$, capacidad $B = t$ y umbral de valor $V = t$. Se pregunta si existe $x \in \{0, 1\}^n$ con suma de pesos a lo más B y suma de valores al menos V. Si existe, la suma de los a_i elegidos es a la vez $\leq t$ y $\geq t$, luego es igual a t. Si algún subconjunto suma t, ese mismo subconjunto cumple las dos desigualdades. La función copia la lista una vez: es $O(n)$. Queda

$$\textsc{Knapsack} \leq_p \text{decisión de la mochila } 0/1.$$

La aceleración de la cadena es esa mochila con otros nombres. El peso es el costo c_i, el valor es la reducción d_i y la capacidad es el presupuesto B. La ecuación (2) mostró que minimizar la duración de la serie es maximizar la suma de reducciones, porque la suma de las duraciones normales es constante. La decisión «¿se puede dejar la cadena en duración a lo más D?» se traduce, restando esa constante, en «¿se puede conseguir reducción al menos igual a $\sum t_i - D$?». No hay un truco de más: es el mismo problema. Como la decisión de la mochila es NP-completa, la decisión de la aceleración también lo es, y la versión de optimización, que pide el subconjunto y no solo el sí o el no, es NP-difícil.

La cadena de reducciones que se puede sostener es, entonces,

$$\textsc{Exact Cover} \leq_p \textsc{Knapsack} \leq_p \text{mochila } 0/1 \leq_p \text{aceleración de la cadena}. \tag{4}$$

Las dos últimas flechas se verifican con el mapeo de arriba. La primera se verifica con la codificación en base β, que es la idea de la prueba de Karp.

El enunciado da un paso más, y el sentido de la reducción vuelve a importar. Un flujo con etapas en paralelo es un grafo dirigido acíclico. La cadena es un caso particular: un camino, que es un grafo dirigido acíclico. Si el problema general se pudiera resolver en tiempo polinomial, el de la cadena también, porque bastaría entregarle ese camino. Como

la cadena ya es NP-difícil, el flujo general también lo es. Formalmente, la aceleración de la cadena $\leq_p$ la aceleración de un flujo cualquiera.

La cobertura de auditoría es el otro problema de la lista de Karp: Set Covering es NP-completo. No hace falta reducirlo a la mochila para saberlo. El algoritmo del repositorio no lo resuelve de forma exacta en el caso general. Usa el voraz que en cada paso se queda con la interacción que cubre más categorías todavía pendientes, y solo en Python, y solo si $n \leq 25$, calcula el óptimo por fuerza bruta para medir el error.

La garantía clásica, que es la que pide el enunciado, es de Johnson [2] y, para la versión ponderada, de Chvátal [1]. Si V es el conjunto que devuelve el voraz y OPT es una cobertura mínima,

$$|V| \leq H_r \cdot |\mathrm{OPT}|, \qquad H_r = \sum_{k=1}^{r} \frac{1}{k} = 1 + \frac{1}{2} + \cdots + \frac{1}{r}. \tag{5}$$

H_r está entre $\ln r$ y $\ln r + 1$.

La razón del argumento es corta. Cuando faltan k categorías, los $|\mathrm{OPT}|$ conjuntos de una solución óptima todavía las cubren, así que alguno cubre al menos $k/|\mathrm{OPT}|$ de las que faltan. El voraz cubre por lo menos eso. Si se reparte 1 entre las categorías cubiertas en ese paso, a cada una le toca a lo más $|\mathrm{OPT}|/k$. Sumando sobre el orden en que se van cubriendo, desde $k = r$ hasta $k = 1$, sale $|V| \leq |\mathrm{OPT}| \cdot H_r$.

Hay una cota más fina que el código sí calcula: $|V| \leq H_d \cdot |\mathrm{OPT}|$, con d el tamaño del conjunto más grande y $H_d \leq H_r$. En las instancias generadas $d \leq 5$, así que

$$H_d \leq 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} = 2.283,$$

mientras H_r crece con r. Comprobar el cociente contra H_d es más exigente que comprobarlo contra H_r, y es lo que hacen los dos programas al imprimir las cotas armónicas.

4. Eficiencia en tiempo y en espacio

Python y C++ implementan la misma recurrencia y el mismo voraz. La función T no cambia con el lenguaje: cambia lo que cuesta cada operación contada como constante. Por eso las dos columnas de cada algoritmo se deducen juntas, y después se dice dónde se separan. Los programas ya cronometran solo el núcleo. En la mochila el reloj envuelve la reserva de la tabla y su llenado, no la lectura del JSON ni el *backtracking*. En el *set cover* el reloj envuelve el bucle voraz, no la prueba de factibilidad ni la escritura del CSV. Las mediciones de la sección 6 usan ese mismo corte, así que se comparan con esta cuenta y no con el tiempo de disco.

4.1. Mochila 0/1

La programación dinámica parte del archivo `0_1_knapsack.cpp` del repositorio The Algorithms [4]. De ahí se tomó la tabla y la recurrencia clásica. En este proyecto el peso del objeto es el costo c_i y el valor es la reducción d_i, y la misma recurrencia está escrita en C++ y en Python. El generador de instancias, la simulación de la cadena y el *set cover* son de este trabajo.

La tabla tiene una fila por etapa y una columna por cada presupuesto entre 0 y B. La recurrencia, con $DP[0][b] = 0$, es

$$DP[i][b] = \begin{cases} \max\big(DP[i-1][b],\ DP[i-1][b-c_i] + d_i\big) & \text{si } c_i \le b, \\ DP[i-1][b] & \text{si } c_i > b. \end{cases} \tag{6}$$

Cada celda mira solo la fila anterior.

1. Crear la tabla escribe un 0 en cada celda. Hay $(n+1)$ filas y $(B+1)$ columnas, luego $(n+1)(B+1)$ escrituras. Eso ya es $\Theta(nB)$.
2. El bucle de las etapas corre de $i = 1$ hasta $i = n$: n vueltas.
3. Por cada etapa, el bucle del presupuesto corre de $b = 0$ hasta $b = B$: $B + 1$ vueltas.
4. El cuerpo de ese bucle no depende de n ni de B. Compara c_i con b y asigna un entero. Si el costo cabe, el entero es el máximo entre no tomar la etapa y tomarla. Son unas pocas sumas y una comparación. Se cuenta como $\Theta(1)$.
5. El producto es $n \cdot (B+1) \cdot \Theta(1) = \Theta(nB)$. Junto con la reserva, el fragmento cronometrado queda
$$T(n, B) = \Theta(nB), \qquad S(n, B) = \Theta(nB). \tag{7}$$
6. Reconstruir la solución recorre las etapas de la n a la 1 y, por cada una, compara dos celdas. Son $\Theta(n)$ operaciones. No están dentro del cronómetro. Aunque se sumen, $\Theta(nB) + \Theta(n) = \Theta(nB)$ en cuanto $B \ge 1$. No cambian la función.

Esto es pseudo-polinomial, y la diferencia es la que el curso pide justificar. $\Theta(nB)$ es polinomial en el *valor* de B. No es polinomial en el tamaño de la entrada si B se mide en bits. Con $|B| = \Theta(\log_2 B)$, se tiene $B = \Theta(2^{|B|})$ y por tanto

$$T = \Theta\big(n \cdot 2^{|B|}\big),$$

exponencial en el largo de la codificación de B. No hay contradicción con la NP-dificultad: la programación dinámica no da un algoritmo polinomial para la mochila. Da un algoritmo eficiente cuando B es pequeño frente a 2^n.

En la familia concreta del generador la conclusión práctica es otra, y hay que escribirla aparte para no mezclarla con el peor caso. Cada costo está entre 5 y 18, así que la suma de costos es $\Theta(n)$ y B, que es el 35 % de esa suma, también es $\Theta(n)$. Sustituyendo, $T(n, B) = \Theta(n \cdot n) = \Theta(n^2)$ sobre estas instancias. El algoritmo sigue siendo $\Theta(nB)$ para un B arbitrario. Sobre los JSON que produce el generador, la curva contra n tiene que verse cuadrática, y la curva contra el producto $n \cdot B$ tiene que verse lineal. La sección 6 contrasta exactamente eso.

Espacio. El *backtracking* compara $DP[i][b]$ con $DP[i-1][b]$, así que la implementación guarda la tabla completa. $S(n, B) = \Theta(nB)$ enteros, más $\Theta(n)$ para las etapas y el vector de decisiones. Los dos lenguajes están en esa clase. No están en el mismo número de bytes.

En C++ cada celda es un `int` de 4 bytes. La tabla ocupa $4(n+1)(B+1)$ bytes, más el encabezado de cada una de las $n+1$ filas, porque es un vector de vectores y no un solo bloque. En Python cada celda es un objeto entero. En CPython un entero pequeño ocupa 28 bytes, y la lista guarda un puntero de 8 bytes hacia él. Si cada celda fuera un objeto

distinto, la tabla andaría por $36(n+1)(B+1)$ bytes, unas nueve veces la de C++. En la práctica muchas celdas valen 0 o repiten la misma reducción, y CPython comparte ese objeto, así que el pico medido queda por debajo de esa cota. Sigue siendo $\Theta(nB)$.

4.2. Set cover voraz

Sea d_i el número de categorías de la interacción i, y $d = \max_i d_i$.

1. Cada elección cubre al menos una categoría nueva, así que el ciclo exterior da a lo más r vueltas. Cada interacción se elige a lo más una vez, así que también da a lo más n vueltas. El número de vueltas es a lo más $\min(n, r)$.
2. En cada vuelta se miran las n interacciones. C++ se salta las que ya marcó. Eso no mejora el peor caso: en las primeras vueltas casi ninguna está marcada y el recorrido sigue siendo $\Theta(n)$.
3. Por cada interacción se cuentan las categorías suyas que siguen pendientes. En el peor caso se inspeccionan $d_i \le r$ elementos, y cada consulta a un conjunto hash es $\Theta(1)$ en promedio. Una interacción cuesta $O(r)$ y una vuelta cuesta $O(nr)$.
4. El producto de las vueltas por el costo de una vuelta es $O\big(\min(n, r) \cdot nr\big)$. Como $\min(n, r) \le r$,
$$T(n, r) = O(nr^2). \tag{8}$$

El comentario del código en C++ escribe $O\big(n \cdot r \cdot \min(n, r)\big)$. Es la misma cuenta, un poco más ajustada, y sigue siendo $O(nr^2)$. Es la función del enunciado.

Otra vez hay que separar el peor caso de la familia que se mide. En el generador $r = \lfloor 0.8n \rfloor = \Theta(n)$ y $d_i \le 5$. Contar las categorías nuevas de una interacción es $O(1)$, no $O(r)$, y hacen falta $\Theta(n)$ vueltas porque cada una aporta a lo más 5 categorías. Sobre estos JSON el tiempo del voraz es $\Theta(n^2)$, no el $\Theta(n^3)$ que saldría de sustituir $r = \Theta(n)$ en el peor caso $O(nr^2)$ sin usar que los conjuntos son chicos. La gráfica contra n debe parecerse a una cuadrática, y tiempo$/n^2$ debe quedarse relativamente plano. Si los conjuntos pudieran tener tamaño proporcional a r, la predicción volvería a ser $O(n^3)$.

Espacio. Se guardan los n conjuntos y el conjunto de categorías pendientes. $S(n, r) = \Theta(n + \sum d_i + r)$. En el generador la suma de los d_i es a lo más $5n$ y r es $\Theta(n)$, así que $S = \Theta(n)$. No es el término que domina: al lado de la tabla de la mochila, el voraz casi no ocupa memoria.

Python, a diferencia de C++, vuelve a intersectar las interacciones ya elegidas. La intersección sale vacía y no cambia la respuesta, pero hace trabajo de más. Sigue dentro de $O(nr^2)$. Es una diferencia de constante, no de función. C++ marca el índice y no lo recorre otra vez.

4.3. El óptimo de set cover, solo en Python

Para $n \le 25$, Python recorre las combinaciones de k interacciones, con k desde 1 hasta n, y se detiene en la primera que cubre el universo. En el peor caso mira los $2^n - 1$ subconjuntos no vacíos. Armar la unión de un subconjunto de tamaño k cuesta proporcional a k por

el tamaño de cada conjunto. Como

$$\sum_{k=0}^{n} \binom{n}{k} k = n \cdot 2^{n-1},$$

queda $T(n) = O(nr\,2^n)$. El espacio auxiliar es $O(n + r)$: se genera una combinación a la vez. Por eso el corte está en 25. No entra en las gráficas de crecimiento, porque C++ no tiene esta búsqueda y porque a partir de ahí el tiempo se va con 2^n, no con las familias de cientos o miles de etapas que sirven para ver $\Theta(nB)$ y $\Theta(n^2)$.

5. Qué implementación conviene optimizar

Optimizar no cambia la clase. Compilar con `-O3` no convierte $\Theta(nB)$ en $\Theta(n)$, y reescribir el mismo bucle en C++ tampoco. Las dos implementaciones tienen la misma $T(n, B)$, la misma $T(n, r)$ y la misma S en notación Θ. La pregunta útil es dónde una hora de trabajo baja más el tiempo que de verdad se mide.

Ese lugar es el llenado de la tabla de la mochila en Python. Es el único fragmento con $\Theta(nB)$ operaciones elementales, una detrás de otra, y en estas instancias B crece con n, así que es también el fragmento $\Theta(n^2)$ más pesado. Cada celda en Python pasa por el intérprete: busca el tipo, ejecuta varios *bytecodes* y escribe un objeto. La misma celda en C++, compilada con `-O3`, es una comparación y una escritura sobre un entero. El voraz no compite con eso en volumen de operaciones, y además la parte cara de Python, la intersección de conjuntos, ya corre en C dentro del intérprete.

En las mediciones de la sección 6 la mochila en Python tarda entre 92 y 115 veces lo que tarda en C++, según el tamaño. En el *set cover* el factor va de 1.2 a 2.5. El hueco grande está en la tabla, no en el voraz. Esos factores son los de la primera máquina. En la segunda, la subsección 6.3, el hueco de la mochila sigue siendo de ese orden.

Por eso, si hay que elegir un código para optimizar, es la mochila en Python. Bajar la constante de C++ —una sola fila de memoria en lugar de un vector de vectores, por ejemplo— mueve un programa que ya está cerca de la máquina. Quitar el intérprete de ese doble bucle, o simplemente usar la versión en C++ que ya está escrita, mueve el tiempo en el factor que se ve en la tabla. La memoria empuja en el mismo sentido: Python guarda decenas de bytes por celda y C++ guarda cuatro. En n grande la tabla de Python puede llenar la memoria antes de que el tiempo sea el problema.

C++ se queda como referencia de lo que ya está medido: misma función de eficiencia y una constante mucho menor. Afinarlo más mueve un programa que ya está cerca de la máquina, con tiempos de milisegundos. La misma mejora, puesta en Python, cae sobre tiempos de segundos.

La decisión de esta entrega es seguir la optimización en Python. El objetivo que viene en el proyecto es incorporar heurísticas: cobertura ponderada, presupuesto conjunto y ruta crítica en el grafo. Una heurística baja la cantidad de trabajo. En Python esa baja se nota en el tiempo medido, porque el punto de partida de la mochila ya es entre 92 y 115 veces más lento que C++. En C++ el mismo algoritmo ya está en unos pocos milisegundos, y la misma mejora relativa mueve un número chico, fácil de confundir con el ruido de la medición. Si más adelante una heurística deja de tener su costo en el bucle de Python, se vuelve a medir y se revisa la elección.

6. Validación experimental

Se generaron instancias con el generador del repositorio, sin cambiar sus rangos: costo entre 5 y 18, reducción acotada para no dejar la etapa en duración negativa, $r = \lfloor 0.8n \rfloor$ categorías, y $B = \lfloor 0.35 \cdot \sum c_i \rfloor$. Los tamaños fueron

$$n \in \{250, 500, 1000, 1500, 2000, 3000\}.$$

Son seis valores, por encima de los cinco que pide el enunciado, y el mayor es doce veces el menor, suficiente para distinguir una recta de una cuadrática.

Cada tamaño se corrió tres veces sobre el mismo JSON y se reporta la mediana, para que un pico del sistema no dibuje la curva. La semilla es $20261105 + n$. El generador no garantiza que la unión de las etapas cubra las r categorías. Con $r = 0.8n$ y a lo más cinco riesgos por etapa, al crecer n quedan categorías fuera y el programa de *set cover* se detiene. Para poder medir el voraz, cada categoría descubierta se agregó a una etapa elegida al azar. Fueron 3, 7, 12, 15, 16 y 26 categorías, en ese orden de tamaños. Ese retoque no cambia n ni r, y deja los conjuntos del mismo orden de tamaño. Python y C++ leyeron el archivo ya reparado.

En los seis tamaños la reducción óptima de la mochila coincidió en los dos lenguajes, y también coincidió el número de auditorías del voraz. La comparación de tiempos es sobre la misma solución, no solo sobre el mismo archivo.

La primera corrida se hizo en un Intel Core i5-10300H a 2,50 GHz, con 4 núcleos y 8 hilos, y 31,8 GB de RAM. C++ se compiló con `g++` 16.2.0, con `-std=c++17 -O3`. Python es CPython 3.13. No se cronometró la generación del JSON, ni su lectura, ni la escritura de los CSV. Esos tiempos no están en la T deducida en la sección 4. El mismo protocolo, con la misma semilla, se repitió en una segunda máquina; esa corrida está en la subsección 6.3.

6.1. Mochila

n	B	$n \cdot B$	Python (ms)	C++ (ms)	Python / C++
250	1005	251 250	36.16	0.3282	110
500	1989	994 500	142.3	1.237	115
1000	4026	4 026 000	596.7	6.462	92
1500	6114	9 171 000	1474.3	15.94	93
2000	8006	16 012 000	2617.1	27.84	94
3000	12014	36 042 000	6161.2	57.62	107

La figura 1 usa escala logarítmica en el tiempo para que las dos curvas quepan en el mismo eje: C++ queda tan abajo que en escala lineal se ve como una raya en el piso. La figura 2 divide cada tiempo por $n \cdot B$. Si el costo por celda es estable, esa gráfica se aplana. Es la firma de $\Theta(nB)$. Una parábola perfecta contra n no lo sería, porque B no es exactamente proporcional a n: depende de la suma aleatoria de los costos.

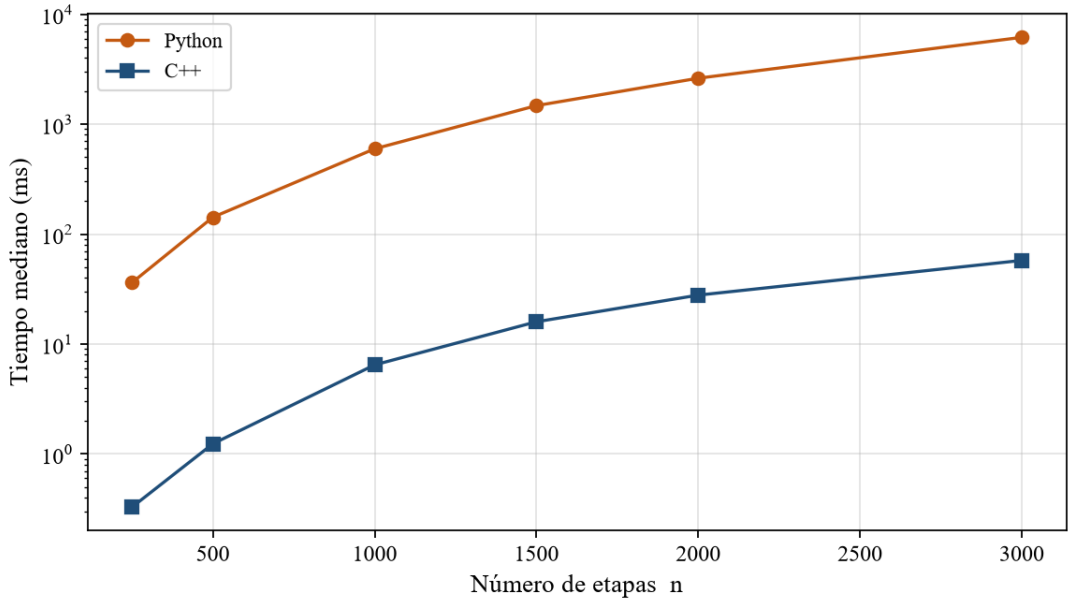


Figura 1: Tiempo mediano de la programación dinámica contra n, en el i5-10300H. Escala logarítmica en el eje vertical.

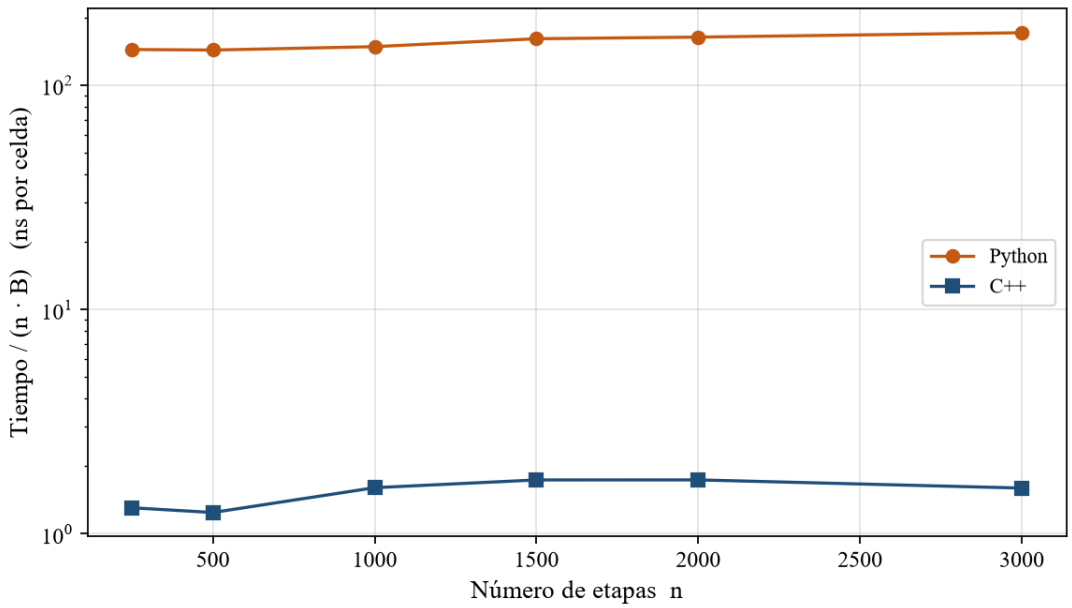


Figura 2: Nanosegundos por celda en el i5-10300H, tiempo dividido por $n \cdot B$, en escala logarítmica. Una curva horizontal es la firma de $\Theta(nB)$. La distancia vertical entre Python y C++ es el factor constante.

Entre el primer y el último tamaño, el tiempo crece como $(n \cdot B)$ elevado a 1.03 en Python y a 1.04 en C++. Un algoritmo $\Theta(nB)$ predice exponente 1. El cociente entre el mayor y el menor costo por celda fue 1.19 en Python y 1.40 en C++. Ese cociente cerca de 1 es lo que se espera si el $\Theta(1)$ del cuerpo del bucle de verdad no depende de n: la curva observada sigue a la función deducida, y la diferencia entre lenguajes es de escala, no de forma.

Para ubicar el espacio: en $n = 3000$, $B = 12014$, la tabla tiene $(3001)(12015) = 36\,057\,015$ celdas. A 4 bytes por entero, C++ usa unos 144 MB solo en celdas. El pico que `tracemalloc` registró en la función de Python, en la primera repetición de ese tamaño, fue 792 MB.

6.2. Set cover

n	r	$\lvert V\rvert$	Python (ms)	C++ (ms)	Python / C++
250	200	72	5.298	3.105	1.7
500	400	146	19.02	15.55	1.2
1000	800	285	87.41	35.34	2.5
1500	1200	433	172.8	90.83	1.9
2000	1600	572	350.8	159.8	2.2
3000	2400	873	774.3	378.0	2.0

La figura 3 es el tiempo contra n. La figura 4 lo divide por n^2, que es la predicción de esta familia, donde cada conjunto cubre a lo más cinco categorías. No se divide por nr^2: esa sería la curva del peor caso, y estas instancias no son el peor caso.

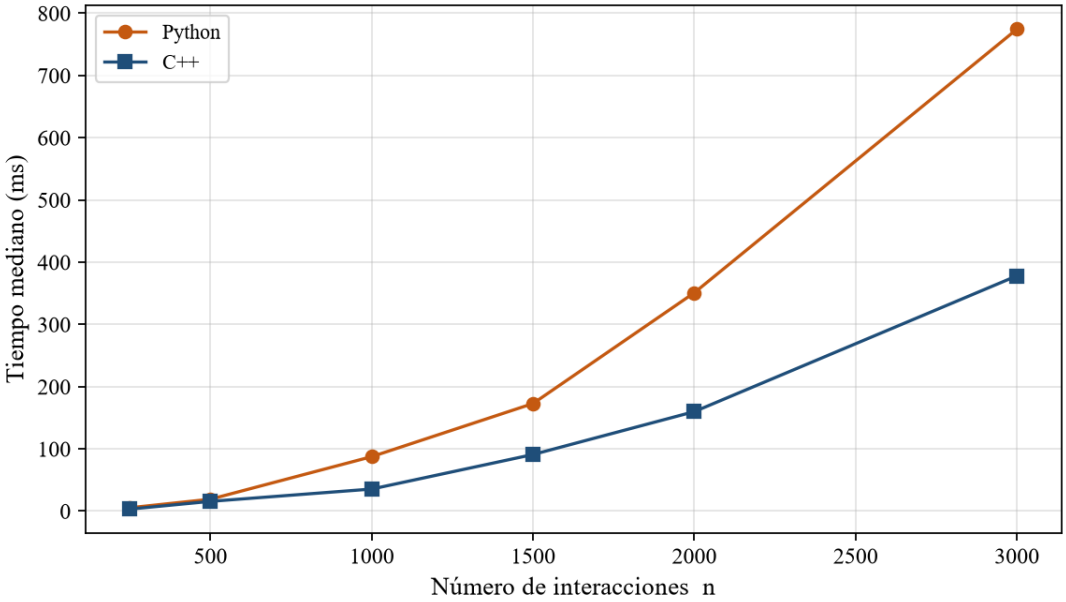


Figura 3: Tiempo mediano del voraz de *set cover* contra n, en el i5-10300H.

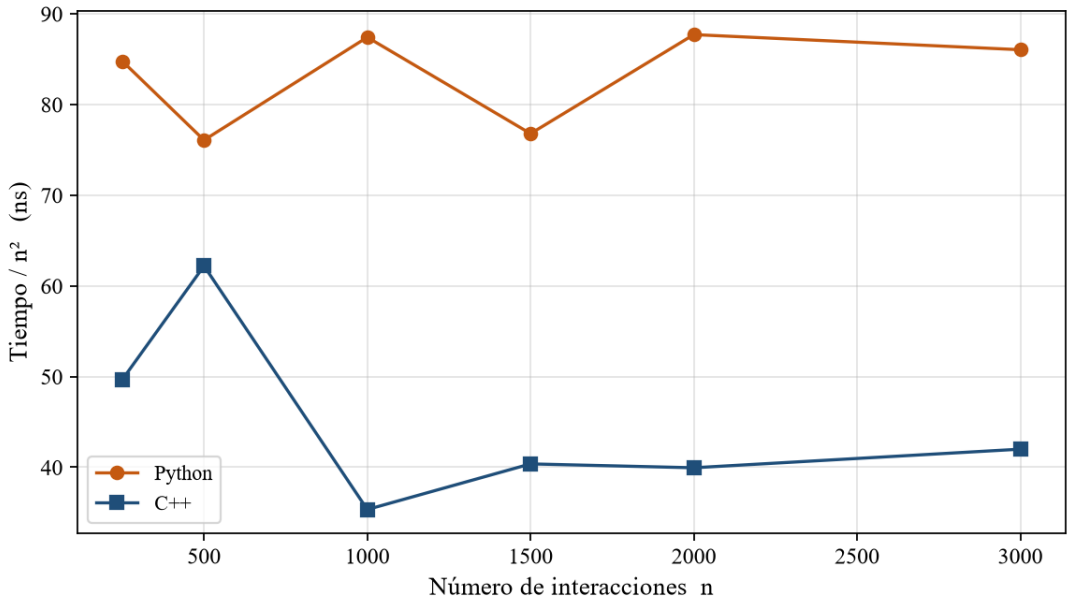


Figura 4: Tiempo del voraz dividido por n^2, en el i5-10300H. Sobre la familia del generador, $\Theta(n^2)$ predice una banda aproximadamente horizontal.

El exponente empírico del tiempo contra n, del primero al último tamaño, es 2.01 en Python y 1.93 en C++. La predicción para esta familia es 2. El valor queda al lado de 2, así que el crecimiento observado acompaña a $\Theta(n^2)$ y no al peor caso $O(nr^2) = O(n^3)$, cuyo exponente sería 3.

6.3. La misma prueba en una segunda máquina

El mismo protocolo, con las mismas semillas $20261105 + n$ y los mismos seis tamaños, se corrió en un AMD Ryzen 5 3500U a 2,10 GHz de base, con 4 núcleos y 8 hilos, caché L1 de 384 kB, L2 de 2,0 MB y L3 de 4,0 MB, y 5,9 GB de RAM. C++ se compiló otra vez con `-std=c++17 -O3`. La semilla fija la instancia: en las dos máquinas coinciden B, la reducción óptima de la mochila y el tamaño $|V|$ de la cobertura voraz. También coinciden las categorías que hubo que agregar para dejar el *set cover* factible: 3, 7, 12, 15, 16 y 26. Lo que cambia es el reloj, no la respuesta.

n	B	$n \cdot B$	Python (ms)	C++ (ms)	Python / C++
250	1005	251 250	62.98	0.3539	178
500	1989	994 500	247.5	4.248	58
1000	4026	4 026 000	1053	13.67	77
1500	6114	9 171 000	2350	37.53	63
2000	8006	16 012 000	4129	66.41	62
3000	12014	36 042 000	10739	125.3	86

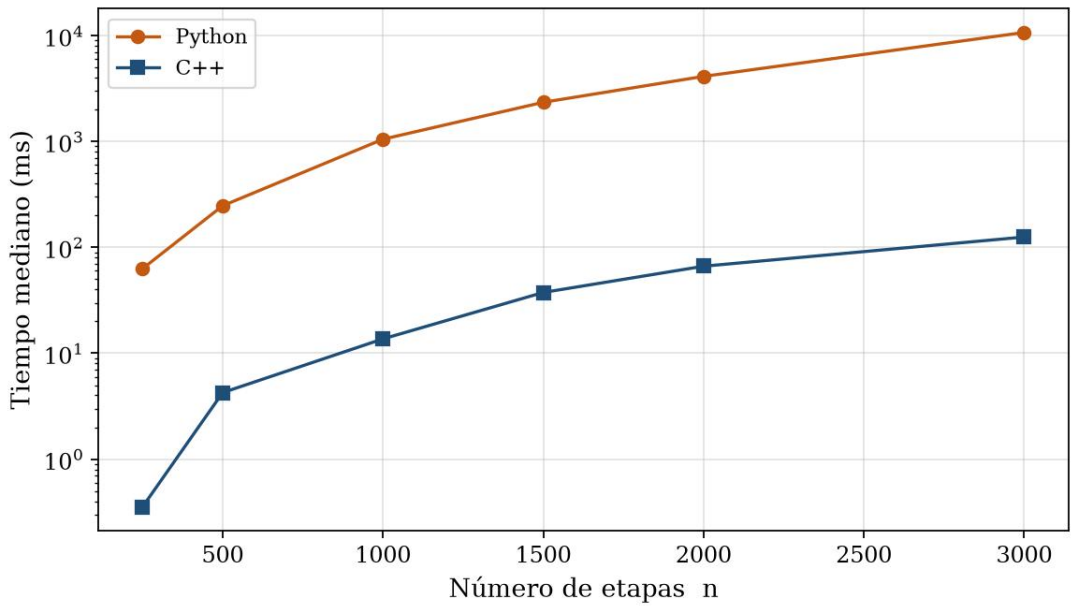


Figura 5: Tiempo mediano de la programación dinámica contra n, en el Ryzen 5 3500U. Escala logarítmica en el eje vertical.

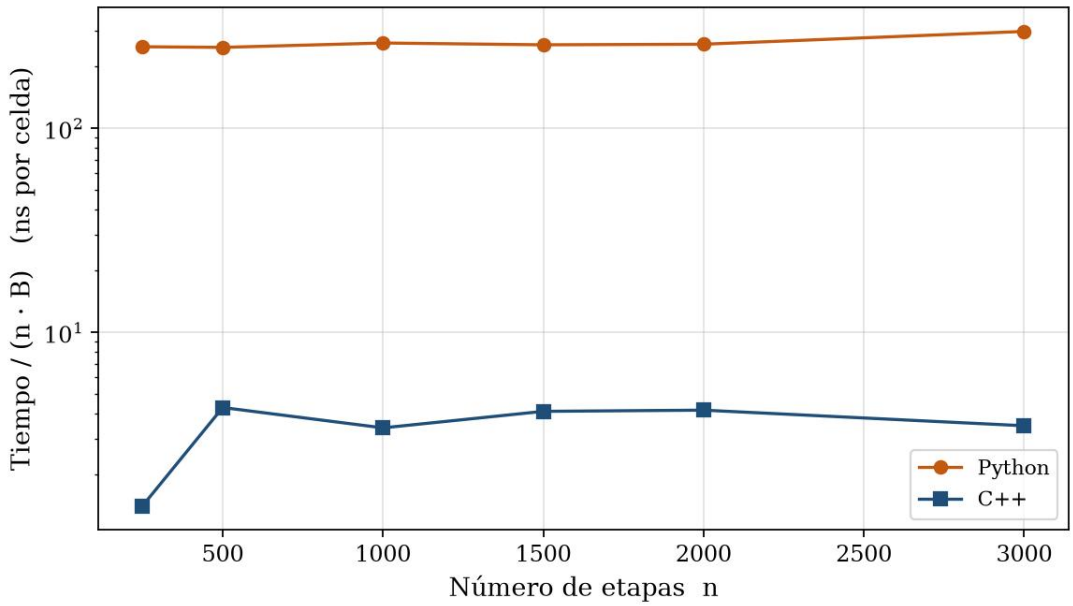


Figura 6: Nanosegundos por celda en el Ryzen 5 3500U. La curva de Python se queda en una banda; el primer punto de C++ queda por debajo del resto.

En la mochila, el tiempo de Python crece como $(n \cdot B)$ elevado a 1.03, el mismo exponente que en el i5. El costo por celda se queda entre 249 y 298 ns: el cociente entre el mayor y el menor es 1.20. En C++ el punto $n = 250$ dura 0,35 ms y sale a 1,4 ns por celda; de $n = 500$ en adelante el costo se queda entre 3,4 y 4,3 ns. Ese primer punto, demasiado corto para el reloj, alarga el exponente de punta a punta hasta 1.18. No es otra función: a partir de $n = 500$ la curva de C++ también es la de $\Theta(nB)$. La máquina cambia la

altura, no el crecimiento. En $n = 3000$ la mochila tarda 10,7 s en Python y 125 ms en C++, frente a 6,16 s y 57,6 ms en el i5-10300H: el Ryzen es más lento en ese tamaño, 1,7 veces en Python y 2,2 veces en C++.

El cociente Python/C++ de la mochila queda entre 58 y 86 a partir de $n = 500$, y sube a 178 en $n = 250$, donde el tiempo de C++ es demasiado corto para un cociente estable. Sigue siendo uno o dos órdenes de magnitud. La decisión de optimizar la tabla en Python no es un accidente del i5.

n	r	$\lvert V\rvert$	Python (ms)	C++ (ms)	Python / C++
250	200	72	7.424	3.656	2.0
500	400	146	26.15	16.21	1.6
1000	800	285	100.6	43.29	2.3
1500	1200	433	233.4	103.9	2.2
2000	1600	572	678.1	245.7	2.8
3000	2400	873	1221	559.0	2.2

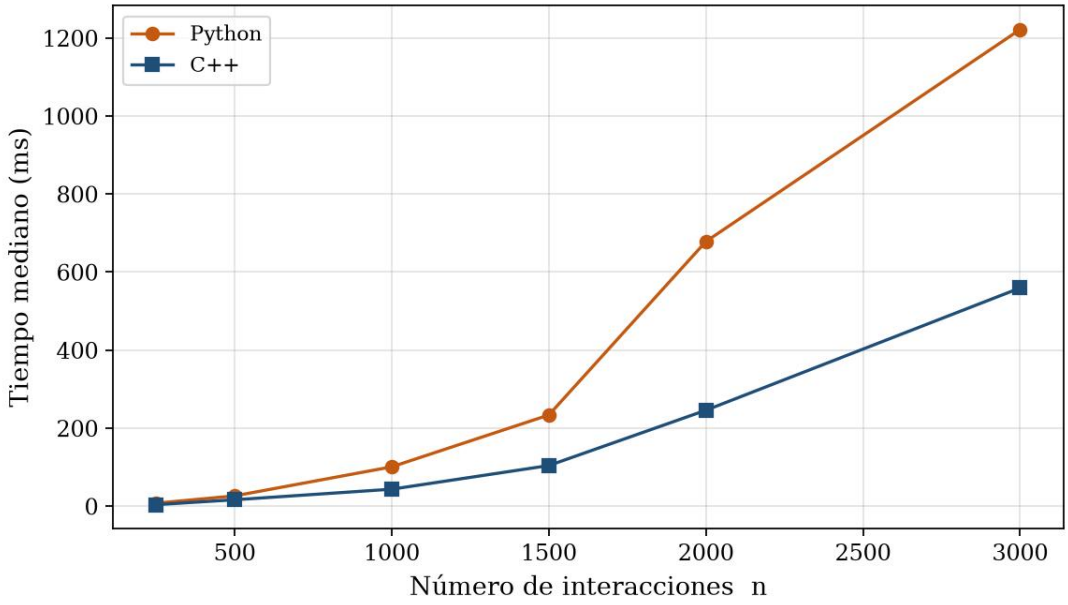


Figura 7: Tiempo mediano del voraz de *set cover* contra n, en el Ryzen 5 3500U.

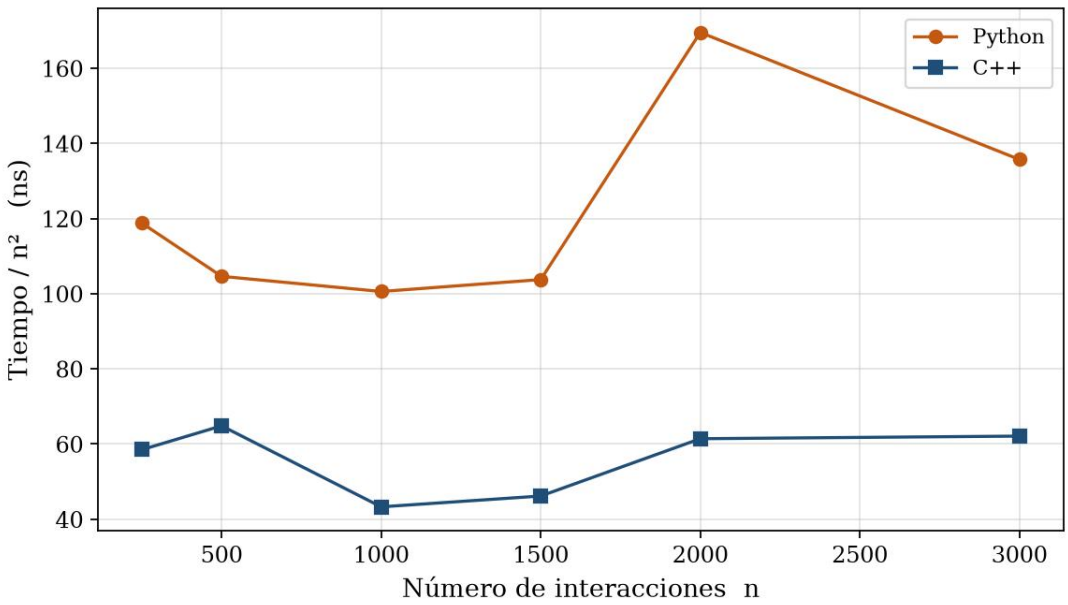


Figura 8: Tiempo del voraz dividido por n^2, en el Ryzen 5 3500U. La banda se mantiene; el pico de Python en $n = 2000$ no cambia el exponente.

En el *set cover* de esta familia los exponentes contra n son 2.05 en Python y 2.02 en C++. La predicción sigue siendo 2, no 3. En la figura de tiempo/n^2 hay un pico de Python en $n = 2000$; no convierte la curva en cúbica. El cociente entre lenguajes queda entre 1.6 y 2.8, del mismo orden que en el i5. En $n = 3000$ el voraz tarda 1,22 s en Python y 559 ms en C++, frente a 774 ms y 378 ms en la primera máquina: 1,6 y 1,5 veces más lento.

Los 5,9 GB alcanzaron para $n = 3000$. La tabla de Python es la que llenaría la memoria primero si n siguiera creciendo: en el i5 el pico de esa función ya fue 792 MB, y aquí la RAM libre es mucho menor. El voraz, con $S = \Theta(n)$, no compite con esa tabla.

7. Cierre

La aceleración de la cadena es la mochila 0/1, y es NP-difícil porque la suma de subconjuntos de Karp se reduce a ella por un mapeo que se comprueba en una pasada, y porque Karp ya había reducido Exact Cover a esa suma de subconjuntos. La programación dinámica la resuelve en $\Theta(nB)$ tiempo y $\Theta(nB)$ espacio. Eso es pseudo-polinomial: polinomial en el valor de B y exponencial en sus bits. En las instancias del generador B es $\Theta(n)$, así que el tiempo contra n es $\Theta(n^2)$, y las mediciones siguen a $n \cdot B$.

La auditoría es *set cover*, NP-completo en el mismo artículo de Karp. El voraz queda dentro de H_r veces el óptimo, con $T(n, r) = O(nr^2)$ en el peor caso y $\Theta(n^2)$ en la familia generada, que es lo que muestra el tiempo dividido por n^2. Python y C++ tienen las mismas funciones. C++ queda como referencia rápida de lo ya medido. Las heurísticas que siguen en el proyecto se trabajan en Python, porque ahí una mejora del algoritmo se nota en el tiempo: el punto de partida está en segundos, y en C++ ya está en milisegundos.

La segunda máquina, un Ryzen 5 3500U con 5,9 GB de RAM, repite las mismas soluciones y la misma forma de las curvas. Solo sube la constante de tiempo. El hueco entre Python

y C++ en la mochila sigue siendo grande, así que la elección de optimizar en Python no depende del procesador en el que se midió la primera tabla.

Referencias

[1] Chvátal, V. (1979). A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3), 233–235.

[2] Johnson, D. S. (1974). Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3), 256–278.

[3] Karp, R. M. (1972). Reducibility among combinatorial problems. En R. E. Miller y J. W. Thatcher (Eds.), *Complexity of Computer Computations* (pp. 85–103). Plenum Press.

[4] The Algorithms. 0/1 Knapsack. En *C-Plus-Plus*. https://github.com/TheAlgorithms/C-Plus-Plus/blob/master/dynamic_programming/0_1_knapsack.cpp